FinalSay - Final Project Report

Evidence snapshot: 12 November 2026 (Asia/Kolkata)
Scope: Current source tree, local SQLite database, repository tests, build configuration, and deployment files. This report describes the code and data observed in this checkout. It does not treat planned deployment, historical reports, or a visible UI as proof of a currently working external service.

Executive summary

FinalSay is a notice verification prototype for educational institutions. It collects notices from explicitly supported official portals, extracts and redacts notice text, retrieves relevant current official notices for a student's submission, and classifies the relationship. Results can include a rationale, model score, evidence, source link, and a human-review path. Local SHA-256, Merkle proofs, and an append-only hash chain provide tamper evidence. A Solidity contract and Polygon Amoy adapter are implemented and locally tested, but the current data contains only local anchors; no public chain deployment or transaction is evidenced.

The current database is internally consistent and contains **2,284 official notice rows**: **2,164 current** and **120 historical**. Three selected sources are active: VTU (720 current rows), University of Delhi Examination (415), and University of Mumbai (990). Older Thapar (24 current) and IIT Bombay (15 current) records remain in the database but are disabled; NIT Jalandhar has a disabled source profile but no notices. All 2,164 selected-source current notices have a publication-date value and no duplicate current source keys were found in the audit query.

The source code and automated verification are substantial: the backend suite passes **202 tests**, the React/TypeScript production build passes, and Foundry passes **4 contract tests**. However, this database snapshot has **zero user accounts, zero student submissions, zero comparison edges, and zero human reviewer annotations**. Therefore, production-data student and reviewer outcomes are not demonstrated by this snapshot. The semantic pipeline and cross-URL revision scenario are covered by code/tests and earlier isolated-database evidence, but an empty `relation_edge` table means no result from a student path is currently persisted here.

The source poll audit has 269 recorded runs: 49 `ok`, 145 `partial`, and 75 `error`. All 49 fully `ok` runs are for VTU. The three selected portals have substantial current rows and poll history, but the latest database runs for VTU, Delhi, and Mumbai are errors (DNS resolution and bad file descriptor are among recorded errors). Scheduled polling is configured for every two hours. Its existence does not guarantee continuous upstream access or complete source coverage.

Assessment: FinalSay is a strong locally testable prototype and a plausible supervised pilot candidate after source recovery and operational controls. It is not a production-ready, institutionally endorsed verification service. No cloud deployment, public blockchain transaction, live human benchmark, or complete source backfill is established in the current evidence.

1. Project purpose

Students receive consequential announcements through institutional sites, PDFs, screenshots, WhatsApp, Telegram, and forwards. A copied notice can be altered, taken out of context, contradicted by a later update, or circulated after it has been replaced. This can cause missed examinations, incorrect deadlines, fraudulent fee payments, and confusion about admissions or recruitment.

FinalSay is designed to compare a student's submitted notice with current official notices from the same institution. Unlike a conventional fact-checking site, it focuses on a structured record of official institutional sources, version history, candidate retrieval, temporal relationships, and provenance evidence. The intended

users are students, institutional issuers, source administrators, and reviewers. The same design can apply to colleges, universities, examination boards, and recruitment bodies, provided each source is explicitly onboarded and tested.

2. User journey and implemented features

1. A student pastes text or submits a PDF/image.
2. The API extracts text, derives issuer/date/deadline/audience/action fields, redacts selected personal details, and stores a SHA-256 fingerprint.
3. Candidate retrieval scopes official notices to the selected institution and ranks them using local semantic embeddings (with lexical/title signals).
4. A local NLI comparison model evaluates related claims and looks for evidence of consistency, contradiction, supersession, correction, extension, or cancellation.
5. Low-signal comparisons can be gated to `unresolved` and routed to reviewer queue records. The result API returns the best candidate, model score, rationale, and evidence.
6. A student can inspect the result, candidate notice, source URL, versions, and provenance status.

The UI contains student submission, result, notice-list, and alert pages; reviewer queue, case, and benchmark pages; Admin source and provenance pages; and issuer publishing. Public registration assigns `student`. Privileged accounts are provisioned through operator tooling, and issuer publishing is scoped by an Admin-managed institution membership. The repository does not contain a full general-purpose Admin dashboard, user self-service profile management, or email/password reset workflow.

The current local DB has no student accounts or submissions, so this report treats student-result and reviewer outcomes as automated-test/integration capabilities rather than live user activity. See Feature Implementation Matrix for status per module.

3. Verification categories and limits

The stored relationship taxonomy is seven labels: `consistent`, `contradictory`, `superseded`, `corrected`, `extended`, `cancelled`, and `unresolved`. The four primary user-facing outcomes are the first three plus `unresolved`.

- **Consistent:** relevant claims are supported by the official notice (semantic NLI entailment path).
- **Contradictory:** relevant claims conflict, without sufficient supported update/replacement context.
- **Superseded:** conflicting event details appear with revision/reschedule/replacement evidence; the system should identify the current official notice replacing the old one.
- **Unresolved:** evidence is weak, candidate retrieval is unavailable, or the score falls below a configured gate; the case can be routed to a reviewer.
- **Corrected / extended / cancelled:** additional relationships handled by model cues and the relationship taxonomy.

The configured model is `sentence-transformers/all-MiniLM-L6-v2` for embedding retrieval and `cross-encoder/nli-deberta-v3-small` for pairwise NLI. The scores are comparative model outputs, not calibrated probabilities. A legacy Hugging Face zero-shot mode and deterministic mock mode exist for optional/test paths; the mock mode is not the semantic production model. No hosted LLM API integration is implemented in the current requirements/configuration.

An update at the *same* source key (commonly an unchanged portal URL whose file bytes changed) creates a linked version and retains the prior row. Many institutions publish a revision at a **new URL**. That new URL is stored as a distinct notice; semantic candidate retrieval can compare it to an old student notice, but the

scraper does not universally create a durable notice-to-notice revision link across URLs. Version history and cross-URL semantic matching are separate mechanisms.

4. Data and source snapshot

The SQLite database at `backend/finalsay.db` passed `PRAGMA integrity_check`. Its file size was 71,667,712 bytes (about 68.3 MiB). Tables show 2,284 official notices, no submission notices, no users, 45 benchmark pairs with zero annotations, zero relation edges, zero review cases, 105 local Merkle roots, 54,516 Merkle proof rows, and 105 local anchor-chain blocks. All Merkle roots in the DB are `anchor_kind=local`; none is recorded as Polygon. The benchmark corpus is persisted, but has no human labels and therefore no current Cohen's Kappa result.

Institution	Source state	Current	Historical	Notes
VTU	Active and polling	720	46	Selected official source
University of Delhi Examination	Active and polling	415	7	Selected official source
University of Mumbai	Active and polling	990	33	Selected official source
Thapar Institute of Engineering and Technology	Disabled	24	26	Retained legacy source records
IIT Bombay	Disabled	15	8	Retained legacy source records
NIT Jalandhar	Disabled	0	0	Source profile/run history only
Total	**3 active profiles**	**2,164**	**120**	**2,284 official rows**

Current selected-source extraction quality: 1,338 `extracted`, 269 `listing_only`, 1 `unextractable`, and 517 `unknown`. `unknown` is not proof of document-level extraction, and listing-only records contain catalog metadata rather than verified document text. No missing publication-date values or duplicate current non-null source keys were found among the three selected sources at this snapshot.

See Web Scraping Capabilities for domain arithmetic, adapter behavior, and poll results.

5. Integrity and provenance

Notice rows retain normalized content SHA-256 hashes and source fields. Daily Merkle roots aggregate notice fingerprints and individual proof rows support integrity verification. A local append-only hash chain links anchor blocks. These are useful tamper-evidence mechanisms, but a local database administrator can alter both application data and local proofs unless the root has been anchored outside the database.

`contracts/FinalSayAnchor.sol` implements an owner-controlled append-only root registry recording the root, batch ID, issuer-set identifier, publisher and timestamp; it rejects duplicate/empty identifiers and unauthorized publishing. The Web3 adapter validates chain ID 80002 and waits for a successful receipt plus contract read-back. Contract tests run locally. Current DB roots are local-only, no transaction hash/receipt or contract address is recorded, and no public-chain deployment is claimed. See the Phase 3 report and `docs/POLYGON-AMOY-DEPLOYMENT.md` for previously recorded setup limits; current code alone cannot establish a funded deployment.

6. Testing and implementation status

Checks run for this report on the current checkout:

- Backend: `backend/.venv/bin/python -m pytest finalsay/tests -q -o addopts=''` — **202 passed**, 4 dependency/deprecation warnings, 43.79 seconds.
- Frontend: `cd apps/web && npm run build` — TypeScript and Vite production/PWA build passed.
- Contracts: `cd contracts && ~/.foundry/bin/forge test -vv` — **4 passed, 0 failed, 0 skipped**.
- Database: read-only SQLite `PRAGMA integrity_check` — `ok`.
- Source state: SQLite reads show 2,164 current selected notices, all with publication dates and no duplicate current source keys; latest selected-source scrape outcomes include errors.

Tests run on isolated test databases verify many registration, ingestion, OCR, scraper, comparison, reviewer, provenance, security-config, and migration behaviors. Automated tests do not prove source freshness, portal completeness, OCR accuracy on every scanned document, real human reviewer agreement, or cloud behavior. No reproducible throughput/latency benchmark was found; this report does not claim one.

The status vocabulary in the companion matrix distinguishes **Fully Implemented and Tested**, **Implemented but Partially Tested**, **Implemented but Not Verified**, **Partially Implemented**, **Planned / Not Implemented**, and **Broken / Currently Non-functional**. A green code-level test does not change the current-database evidence: this snapshot has no real users or submissions.

7. Readiness assessment

Strengths: clear module separation; source-specific adapters; role-based APIs; isolated tests; deterministic ingestion/version semantics; semantic retrieval/NLI; source-run observability; local proofs; contract implementation and tests; deployment templates; production-config validation.

Material gaps before a public pilot: restore reliable access to all three selected portals and complete backfills; establish monitoring and operator alerting for repeated poll/extraction failures; validate source-by-source document extraction and completeness; have real students exercise submission and verify outputs; provision real reviewers and collect blind labels before reporting agreement; use a reachable production database and strong secrets; fix the Pages workflow's configured HTTP API URL before using it from an HTTPS page; deploy and observe the full frontend/API stack; decide and document institutional governance and escalation for ambiguous or harmful false positives.

Overall: major prototype engineering is implemented, but current evidence is not sufficient to say the end-to-end real-user product is complete. The best description is **feature-rich prototype, locally tested, operationally unverified on current real users, and not deployed**.

8. Technology stack and hosting

Layer	Actual technology	Current deployment evidence
Languages	TypeScript, Python, Solidity	Source present in repository
Frontend	React, React Router, Vite, PWA plugin	Local development/build configuration; GitHub Pages workflow present
Backend	Python 3.11, FastAPI, Uvicorn, SQLAlchemy	Local API configuration; Docker image and ECS task/workflow templates
Database	SQLite local default; PostgreSQL supported	Current `backend/finalsay.db` is SQLite; production database not established

Layer	Actual technology	Current deployment evidence
ML/OCR	Sentence Transformers, MiniLM, DeBERTa NLI, PyTorch; pypdf, PyMuPDF, Pillow, pytesseract/Tesseract	Models and extraction code present; Tesseract depends on host/runtime install
Blockchain	Solidity 0.8.24, Foundry, Web3.py	Contract and local tests present; no public deployment/transaction
Hosting	Localhost for development; Docker Compose and AWS ECS/Fargate/GitHub Pages workflows prepared	No currently running cloud deployment established in the repository evidence

The Pages workflow currently hard-codes an HTTP API URL, which must be replaced by a verified HTTPS endpoint before relying on that HTTPS frontend deployment. See the Security and Scalability Audit.

For route-by-route and schema detail see Technical Architecture. For audit findings and practical work order see Security and Scalability Audit.